

2.4 Advanced Preprocessing

POS tags, a first look at entities, and why parsing is optional.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

1. When the basic recipe is the wrong tool

Notes **2.2–2.3** get you a bag of tokens. Some jobs need *who did what to whom*, or *which span is a name*.

Example. Find person and organization names in a million company documents. Lowercasing and stop-word removal hide the signal. You want **part-of-speech (POS)** tags first: proper nouns are a cheap clue for names. Then you want **named-entity recognition (NER)**. Full **parsing** is the heavier option when you need syntax, not just tags.

This note is a map, not a tagger from scratch. You will use NLTK or spaCy.

2. POS tagging

Given a sentence, attach a syntactic category to each token:

```
This is a simple sentence
This/DET is/VB a/DET simple/ADJ sentence/NOUN
```

POS is a first step toward syntax, and syntax is a first step toward meaning. A tagger is simpler and faster than a full parser, and often enough:

- features for text classification
- word-sense disambiguation (`water` as noun vs verb)
- a pre-filter for NER (proper nouns)

3. Why tagging is hard

The same string is not always the same tag.

String	Noun	Verb
water	glass of water	water the plants
lie	tell a lie	lie down
wind	a mighty wind	wind down

Time flies like an arrow has several legal analyses. Add words you have never seen, or a noun used as a verb (to google), and a dictionary alone fails.

Buffalo buffalo Buffalo buffalo buffalo buffalo Buffalo buffalo



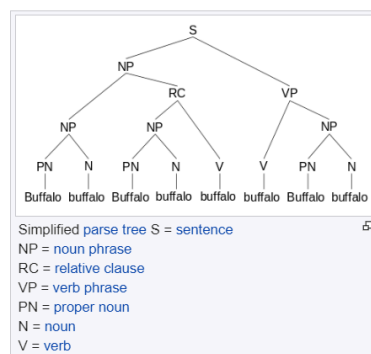
From Wikipedia, the free encyclopedia

"Buffalo buffalo Buffalo buffalo buffalo buffalo Buffalo buffalo" is a [grammatically correct sentence](#) in [American English](#), often presented as an example of how [homonyms](#) and [homophones](#) can be used to create complicated linguistic constructs through [lexical ambiguity](#). It has been discussed in literature in various forms since 1967, when it appeared in [Dmitri Borgmann's](#) *Beyond Language: Adventures in Word and Thought*.

The sentence employs three distinct meanings of the word *buffalo*:

- as a proper noun to refer to a specific place named Buffalo, the city of [Buffalo, New York](#), being the most notable;
- as a verb (uncommon in regular usage) *to buffalo*, meaning "to bully, harass, or intimidate" or "to baffle"; and
- as a noun to refer to the animal, [bison](#) (often called *buffalo* in North America). The plural is also *buffalo*.

An expanded form of the sentence which preserves the original word order is: "Buffalo bison, that other Buffalo bison bully, also bully Buffalo bison."



Other headaches: context, dialect, overlapping categories, and **out-of-vocabulary** words.

4. Penn Treebank

You need a labeled corpus to train or to evaluate a tagger. The **Penn Treebank** (1989–1996) is the English standard you will see cited:

- about 7 million words with POS tags
- millions more with partial or full parses
- spoken transcripts marked for disfluency

Sources include Wall Street Journal, IBM manuals, nursing notes, and telephone speech. The WSJ section is what most off-the-shelf English taggers were measured on.

The tag set is coarse but useful: `NNP` proper noun, `VB / VBD / VBN` verb forms, `JJ` adjective, `DT` determiner, and so on. NLTK's `pos_tag` returns these tags.

5. How taggers are built (high level)

Rules. Start with a dictionary of possible tags per word. Assign every possible tag. Write rules that delete the wrong ones until one tag remains (ENGTWOL-style). You need a large lexicon and a patient linguist.

Probabilistic. For tokens $w_1, \dots, w_n$, find the tag sequence $t_1, \dots, t_n$ with the highest probability. Hidden Markov models and related sequence models are the classic version.

Machine learning / deep learning. Sequence classifiers and neural taggers. They are strong, but they are not automatically cheaper than a good stochastic tagger. Cost still matters.

In this course you do **not** train one from scratch. You call a library:

```
import nltk
from nltk.tokenize import word_tokenize

text = "Charles Spencer Chaplin was born in London England on April 16th 1889"
nltk.pos_tag(word_tokenize(text))
```

spaCy does the same in one pipeline object and is what you want if you also need NER.

6. NER and other span labels

Named-entity recognition labels spans, not only single words:

```
Barack/PER Obama/PER spoke/NON from/NON the/NON White/LOC House/LOC today/NON
```

Typical types: person, organization, location, date, money. Everything else is `O` / none. POS helps; it is not NER by itself. `White House` is two tokens and one entity.

Field segmentation is the same idea on a fixed template: in a classified ad, mark price, size, location; in a bibliography, mark author, title, year.

7. Parsing, at a high level

A **parser** builds a tree (constituency) or a graph of heads and dependents (dependency). That is how you get subject–verb–object without writing a new regex per sentence.

You need a parse when the task is syntactic (relation extraction, some question answering). You do not need one to start a bag-of-words classifier. Parsers are slower and they error on messy web text. Use tags first; parse when the extra structure pays for itself.

8. Practice

1. Tag `I saw her duck` in two different ways. What changes besides the POS of `duck`?
2. Why is lowercasing a bad idea *before* POS tagging?
3. Is “every capitalized word is a person” NER? Give a counterexample.